KRYLOV ITERATION AND GMRES

Feng Chengliang

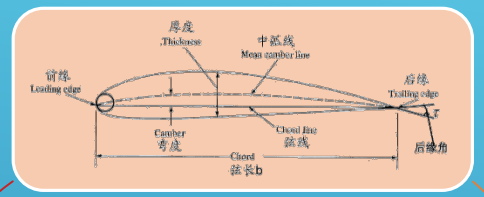
charlon_feng@buaa.edu.cn

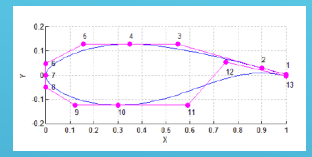
2025.9.17

飞机机翼设计为例

1、机翼参数化

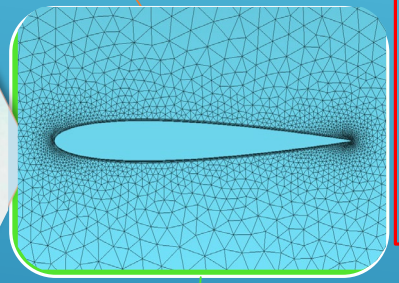
- 插值与逼近





2、网格剖分

- 线性方程组求解

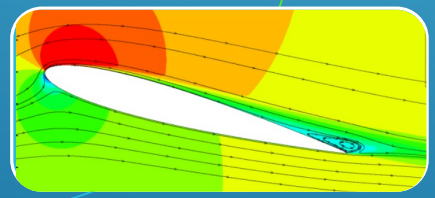




3、流场模拟

- 常微分方程求解
- 线性方程组求解
- 特征值求解

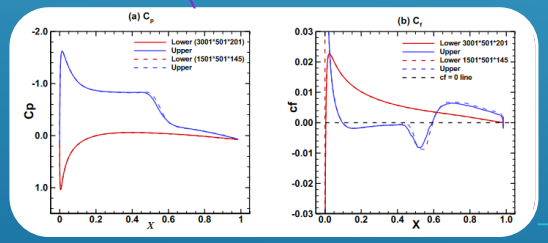
$$\frac{\partial U}{\partial t}+\frac{\partial F}{\partial x}+\frac{\partial E}{\partial y}+\frac{\partial G}{\partial z}=0$$



4、升阻力计算

- 数值积分

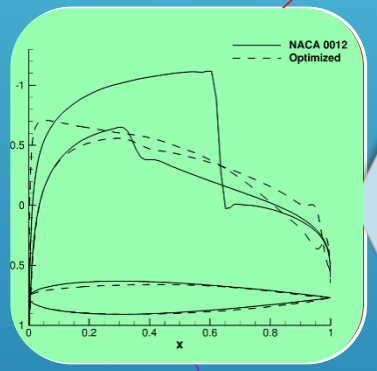
$$N=\oint(-p\cos\theta+\tau\sin\theta)ds$$
$$A=\oint(\tau\cos\theta+p\sin\theta)ds$$



5、设计参数优化

- 非线性方程根
- 优化算法

$$\min f(\boldsymbol{D}_k)=C_d(\boldsymbol{D}_k)+M_k\bullet P(\boldsymbol{D}_k)$$
$$P(\boldsymbol{D}_k)=\Theta_{Cd}+\Theta_{Area}+\Theta_{CST}$$



Section 1 Classical Iterative Methods

$$Ax = b \rightarrow x = Gx + Qb$$

$$x^{k+1} = Gx^{k} + d$$

$$f(x) = 0 \rightarrow x = x - f(x)$$

① $\rightarrow qx = qx - f(x)$

$$Qx = Qx - (Ax - b) = (Q - A)x + b$$

$$Qx_{k+1} = (Q - A)x_k + b$$

② $\rightarrow qx + g(x) = qx - (f(x) - g(x))$

$$Qx + Bx = Qx - \big((A - B)x - b\big) = (Q - A + B)x + b$$

$$(Q + B)x_{k+1} = (Q - A + B)x_k + b$$

③ $\rightarrow qx + kg(x) = qx - k(f(x) - g(x))$

$$(Q + kB)x = Qx - k\big((A - B)x - b\big)$$

$$(Q + kB)x_{k+1} = (Q - kA + kB)x_k + kb$$

Section 1 Classical Iterative Methods

$$x^{k+1} = Gx^k + d$$

- Jacobi iteration

$$\begin{cases} x_1^{(k+1)} = -\frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} + \frac{b_1}{a_{11}} \\ x_2^{(k+1)} = -\frac{a_{21}}{a_{22}}x_1^{(k)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{2n}}{a_{22}}x_n^{(k)} + \frac{b_2}{a_{2n}} \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ x_n^{(k+1)} = -\frac{a_{n1}}{a_{nn}}x_1^{(k)} - \frac{a_{n2}}{a_{nn}}x_2^{(k)} - \cdots - \frac{a_{n,n-1}}{a_{nn}}x_{n-1}^{(k)} + \frac{b_n}{a_{nn}} \end{cases}$$

Jacobi

- Gauss-Seidel iteration

$$\begin{cases} x_1^{(k+1)} = -\frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} + \frac{b_1}{a_{11}} \\ x_2^{(k+1)} = -\frac{a_{21}}{a_{22}}x_1^{(k+1)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{2n}}{a_{22}}x_n^{(k)} + \frac{b_2}{a_{2n}} \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ x_n^{(k+1)} = -\frac{a_{n1}}{a_{nn}}x_1^{(k+1)} - \frac{a_{n2}}{a_{nn}}x_2^{(k+1)} - \cdots - \frac{a_{n,n-1}}{a_{nn}}x_{n-1}^{(k+1)} + \frac{b_n}{a_{nn}} \end{cases}$$

Gauss-Seidel

- SOR iteration
  - **The necessary and sufficient condition for convergence is ρ(G)<1**
  - **If ||G||<1, the method converges**
  - **If A is strictly diagonally dominant, the method converges (0<ω<=1)**

$$\begin{cases} x_1^{(k+1)} = x_1^{(k)} + \omega\left(-x_1^{(k)} - \frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} + \frac{b_1}{a_{11}}\right) \\ x_2^{(k+1)} = x_2^{(k)} + \omega\left(-\frac{a_{21}}{a_{22}}x_1^{(k+1)} - x_2^{(k)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{2n}}{a_{22}}x_n^{(k)} + \frac{b_2}{a_{22}}\right) \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ x_n^{(k+1)} = x_n^{(k)} + \omega\left(-\frac{a_{n1}}{a_{nn}}x_1^{(k+1)} - \frac{a_{n2}}{a_{nn}}x_2^{(k+1)} - \cdots - \frac{a_{n,n-1}}{a_{nn}}x_{n-1}^{(k+1)} - x_n^{(k)} + \frac{b_n}{a_{nn}}\right) \end{cases}$$

SOR

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

2.1 Krylov Subspaces and Projection Methods

Teaching Content:

- **Definition of a Krylov subspace**:

$$\mathcal{K}_m(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \ldots, A^{m-1}r_0\},$$

where $r_0 = b - Ax_0$ is the initial residual.

- **The unifying idea of Krylov subspace methods**: find an approximate solution x_m in $\mathcal{K}_m(A, r_0)$ such that the residual satisfies a certain optimality condition (e.g., the Petrov–Galerkin condition).
- **Two main families of methods**:
  - **CG (Conjugate Gradient method, 1952)**: applicable to symmetric positive definite matrices; maintains conjugate directions through short recurrences, requires only 1 matvec per iteration, and uses $O(n)$ storage.
  - **GMRES (Generalized Minimal Residual method, Saad & Schultz, 1986)**: applicable to general nonsymmetric nonsingular matrices; builds an orthonormal basis via the Arnoldi process and minimizes the 2-norm of the residual, with storage growing with the iteration count.
- **Key insight**: CG exploits symmetry to achieve short recurrences, whereas GMRES makes no symmetry assumption on A and therefore requires long recurrences and more storage. **This is the fundamental trade-off between "generality" and "efficiency".**

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

2.2 Arnoldi 过程（15 分钟）

教学内容：

（一）算法目标：给定 $A \in \mathbb{R}^{n\times n}$ 和初始向量 v_1（通常取 $v_1 = r_0/\|r_0\|_2$），构造一组标准正交基 $\{v_1, v_2, \ldots, v_m\}$ 张成 $\mathcal{K}_m(A, v_1)$。

（二）Arnoldi 算法伪代码

```
Algorithm Arnoldi(A, v_1, m)
    for j = 1, 2, ..., m do
        w = A v_j                       // 矩阵-向量乘
        for i = 1, 2, ..., j do
            h_{i,j} = w^T v_i           // 投影到已有基向量
            w = w - h_{i,j} v_i         // 正交化 (modified Gram-Schmidt)
        end for
        h_{j+1,j} = ||w||_2             // 新方向长度
        if h_{j+1,j} == 0 then          // 已收敛 (happy breakdown)
            m = j; break
        end if
        v_{j+1} = w / h_{j+1,j}         // 归一化得新基向量
    end for
    return V_m = [v_1, ..., v_m], H_{m+1,m} = [h_{i,j}]
```

（三）Arnoldi 过程的核心性质

经过 m 步 Arnoldi，得到分解关系

$$AV_m = V_{m+1}\tilde{H}_m,$$

其中 $V_m \in \mathbb{R}^{n\times m}$ 列正交（$V_m^T V_m = I_m$），$\tilde{H}_m \in \mathbb{R}^{(m+1)\times m}$ 为上 Hessenberg 矩阵（仅 $h_{i,j}$ 在 $i \le j+1$ 时非零）。

这个分解将高维矩阵 A 的作用压缩到低维上 Hessenberg 矩阵 $\tilde{H}_m$ 中，是导出 GMRES 算法的关键。

Algorithm (Arnoldi Process). Let $\beta = \|\boldsymbol{r}^{(0)}\|_2$, $\boldsymbol{v}_1 = \frac{\boldsymbol{r}^{(0)}}{\beta}$. For $j = 1, \ldots, m$:

1. Compute $w = A\boldsymbol{v}_j$
2. For $i = 1, \ldots, j$, compute $h_{i,j} = \boldsymbol{v}_i^T w$, $w = w - h_{i,j}\boldsymbol{v}_j$
3. Set $h_{j+1,j} = \|w\|_2$
4. If $h_{j+1,j} = 0$, stop
5. Otherwise set $\boldsymbol{v}_{j+1} = w/h_{j+1,j}$

Theorem (Arnoldi relation). *The Arnoldi basis satisfies*

$$AV_m = V_{m+1}H_m$$

Where

$$H_m = \begin{vmatrix} \widehat{H_m} \\ h_{m+1,m}e_m^T \end{vmatrix}$$

and $H_m \in R^{m\times m}$ *is upper Hessenberg*

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

2.3 GMRES 算法（15 分钟）

教学内容：

（一）问题转化

设 $x_m = x_0 + V_m y$ $(y \in \mathbb{R}^m)$，则残差

$$r_m = b - Ax_m = r_0 - AV_m y = \beta v_1 - V_{m+1}\tilde{H}_m y,$$

其中 $\beta = \|r_0\|_2$，$v_1 = r_0/\beta$。

利用 V_{m+1} 列正交，残差范数简化为

$$\|r_m\|_2 = \|\beta e_1 - \tilde{H}_m y\|_2,$$

其中 $e_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{m+1}$。

核心结论：GMRES 在第 m 步的最优解等价于求解 $(m+1) \times m$ 的最小二乘问题

$$y_m = \arg\min_{y \in \mathbb{R}^m} \|\beta e_1 - \tilde{H}_m y\|_2.$$

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

（二）上 Hessenberg 最小二乘的求解

关键在于 $\tilde{H}_m$ 的上 Hessenberg 结构允许通过 **Givens 旋转**序列递推求解，避免每步从零开始计算 QR 分解：

1. 在第 j 步 Arnoldi 得到 $\tilde{H}_j$ 的新列后，将已计算的 Givens 旋转应用到新列；
2. 生成一个新的 Givens 旋转 G_j 消去 $\tilde{H}_j$ 的次对角元 $h_{j+1,j}$；
3. 累积旋转到右端向量 βe_1；
4. 检查 $|(\beta e_1)_{j+1}|$（旋转后的最后一个分量）——即当前步的残差范数估计。

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

（三）完整 GMRES 算法

```
Algorithm GMRES(A, b, x_0, m_max, tol)
    r_0 = b - A x_0
    beta = ||r_0||_2; v_1 = r_0 / beta
    for j = 1, 2, ..., m_max do
        // Arnoldi 第 j 步
        w = A v_j
        for i = 1 to j do
            h_{i,j} = w^T v_i
            w = w - h_{i,j} v_i
        end for
        h_{j+1,j} = ||w||_2; v_{j+1} = w / h_{j+1,j}

        // 应用 Givens 旋转到新列
        for i = 1 to j-1 do
            rotate(h_{i,j}, h_{i+1,j}, c_i, s_i)
        end for
        (c_j, s_j) = givens(h_{j,j}, h_{j+1,j})
        h_{j,j} = c_j * h_{j,j} + s_j * h_{j+1,j}
        h_{j+1,j} = 0
        g_{j+1} = -s_j * g_j
        g_j = c_j * g_j

        // 检查收敛
        res_norm = |g_{j+1}|
        if res_norm < tol * ||b|| then
            // 回代求解 y
            solve H_j(1:j,1:j) * y = g(1:j)   (上三角)
            x = x_0 + V_j y
            return x
        end if
    end for
    // 达到最大步数尚未收敛，需重启
    x = x_0 + V_{m_max} y; x_0 = x; goto start
```

Theorem： given A is symmetrical and positive definite, and its eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n > 0$

The sequence gives by the steepest gradient method is convergent

$$\lim_{k\to\infty} x^{(k)} = x^* = A^{-1}b$$

and holds $$\left\|x^{(k)} - x^*\right\|_A \leq \left(\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n}\right)^k \left\|x^{(0)} - x^*\right\|_A$$

$$\|x\|_A \triangleq \sqrt{x^T A x}$$

- If $\lambda_1 >> \lambda_n$, the convergence is very slow

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method

（四）重启策略（Restarted GMRES）

- **问题**：GMRES 存储所有 m 个 Arnoldi 向量（$O(mn)$），m 过大时内存无法承受。
- **解决方案**：使用重启版本 GMRES(m)——每 m 步重启，以当前近似解为新的初始猜测重启 Arnoldi 过程。
- **代价**：重启会丢失超线性收敛信息，对某些问题（如含小特征值簇的矩阵）可能导致停滞。
- **实践建议**：m 通常取 20–50，在内存允许范围内越大越好。

（五）GMRES 的收敛性质

- **精确算术下**：GMRES 至多 n 步收敛到精确解（即 $m = n$ 时 $H_{n+1,n}$ 的最后一行全零，即"happy breakdown"）。
- **收敛速度**：由 A 的谱分布（特征值集合的"聚簇性"）决定。特征值紧密聚拢时收敛快；特征值散布且靠近原点时收敛慢。
- **关键认识**：条件数 $\kappa(A)$ 不是 GMRES 收敛速度的可靠指标——一个条件数大但特征值聚拢的矩阵可能比条件数小但谱散布的矩阵收敛更快。
- **预处理的重要性**：GMRES 的收敛强烈依赖预处理——这就是第 3 学时的核心内容。

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method
- ILU-GMRES method

3.1 预处理的基本概念（10 分钟）

教学内容：

（一）为什么需要预处理

GMRES 的收敛取决于系数矩阵 A 的谱性质。对许多实际工程问题，A 的条件数很大或特征值散布广泛，导致 GMRES 收敛极慢。预处理旨在构造一个矩阵 M（预处理子），使得 $M^{-1}A$（左预处理）或 AM^{-1}（右预处理）具有更优的谱性质。

（二）预处理方式

- **左预处理**：求解 $M^{-1}Ax = M^{-1}b$。残差范数对应预处理后的系统，可能与原始残差有偏差。
- **右预处理**：求解 $AM^{-1}y = b$，$x = M^{-1}y$。GMRES 使用原始残差范数作为停止准则，更安全。GMRES 通常采用右预处理。

预处理后的 Arnoldi 关系变为

$$(AM^{-1})V_m = V_{m+1}\tilde{H}_m.$$

（三）理想预处理子

显然 $M = A$ 是最优的（此时系统被精确求解），但构造 A^{-1} 的代价与原问题相同。实用预处理子需要在**构造代价**与**预处理效果**之间取得平衡。不完全 LU 分解正是实现这种平衡的经典方案。

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method
- ILU-GMRES method

3.2 不完全 LU 分解（20 分钟）

教学内容：

（一）从完全 LU 到不完全 LU

完全 LU 分解将 A 分解为 $A = LU$，其中 L 为单位下三角，U 为上三角。对稀疏矩阵，L 和 U 中会出现大量**填充元**（fill-in）——即原矩阵 A 中为零但分解后变为非零的元素。填充元导致存储和计算爆炸。

不完全 LU 分解（ILU）的核心思想：**在分解过程中有选择地丢弃填充元，保持 L 和 U 的稀疏性，**使得 $M = LU \approx A$ 但构造代价可控。

（二）ILU(0)：零填充级 ILU

ILU(0) 是最简单的版本：**只允许在 A 的非零模式内产生 L 和 U 的非零元**，所有填充元一律丢弃。

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method
- ILU-GMRES method

3.3 ILU 预处理 GMRES 的完整算法（15 分钟）

教学内容：

（一）右预处理 GMRES 算法

```
Algorithm PGMRES(A, b, M, x_0, m_max, tol)
    // M = L*U 为 ILU 预处理子
    r_0 = b - A x_0
    beta = ||r_0||_2; v_1 = r_0 / beta
    for j = 1, 2, ..., m_max do
        // 预处理操作 z = M^{-1} v_j
        Solve L z_tmp = v_j     // 前代 (forward substitution)
        Solve U z = z_tmp       // 回代 (back substitution)

        // 对预处理后的向量做 Arnoldi
        w = A z
        for i = 1 to j do
            h_{i,j} = w^T v_i
            w = w - h_{i,j} v_i
        end for
        h_{j+1,j} = ||w||_2; v_{j+1} = w / h_{j+1,j}

        // Givens 旋转及收敛检查（同上节）
        ...
    end for
    x = x_0 + Z_m y          // Z_m = [z_1, ..., z_m]
    return x
```

SECTION 2 ： KRYLOV ITERATION AND GMRES

- GMRES method
- ILU-GMRES method

与标准 GMRES 的关键区别：

1. 每次 Arnoldi 迭代前需要做一次**前代 + 回代**（$O(\text{nnz}(L+U))$）——这是预处理的主要代价；
2. 收敛所需迭代次数大幅减少（通常降低 1–2 个数量级）；
3. 解的近似解来自 z 的线性组合而非 v。

（二）计算代价分析

操作	每次迭代复杂度
SpMV（Az）	$O(\text{nnz}(A))$
ILU 前代+回代（L, U）	$O(\text{nnz}(L+U))$
Arnoldi 正交化（MGS）	$O(jn)$

预处理开销的权衡：尽管每次 Arnoldi 迭代增加了前代/回代的代价，但迭代次数从上百次降至十次以内，**总计算时间通常大幅降低**。

（三）一个直观理解

ILU 预处理相当于在 GMRES 之前先对 A 做一个"粗粒度的近似求逆"。这个近似逆 $M^{-1} \approx A^{-1}$ 把 A 的特征值映射到 1 附近，使得 GMRES 在一个"几乎已求解"的系统上只需少量修正。

数值可视化：在实例中绘制 ILU 预处理前后的特征值分布对比图——预处理后的特征值应明显聚拢在 1 附近。

SECTION 3 : APPLICATION

- Two-Point

Boundary Value Problem

$$\begin{cases} \frac{d^2u}{dx^2} - (1+x)^2 \frac{du}{dx} - e^{-x}u = f(x), 0 < x < 1, \\ f(x) = \left[1-(1+x)^2\right]e^x - \pi^2 \cos(\pi x) + \pi(1+x)^2 \sin(\pi x) - 1 - e^{-x}\cos(\pi x) \\ u(0) = 2, \quad u(1) = e-1 \end{cases}$$

Excat solution: $u(x) = e^x + \cos(\pi x)$

$$x_0 = 0 < x_1 < \cdots < x_i < \cdots < x_{N-1} < x_N = 1 \qquad x_i = i \cdot h, 0 \le i \le N, h = \frac{1}{N}$$

finite difference equation: $\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} - (1+x_i)^2 \frac{u_{i+1} - u_{i-1}}{2h} - e^{-x_i}u_i = f(x_i),\ 1 \le i \le N-1$

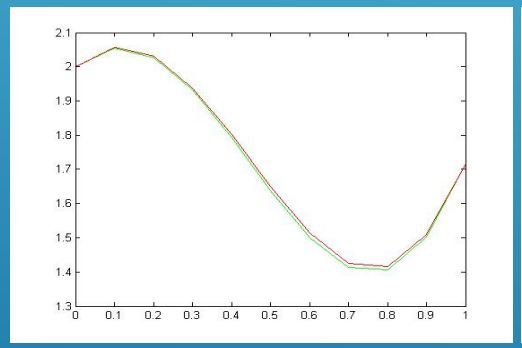
$$AU = F$$

$$\begin{cases} [2 + h(1+x_i)^2]u_{i-1} - [4 + 2h^2 e^{-x_i}]u_i + [2 - h(1+x_i)^2]u_{i+1} = 2h^2 f(x_i),\ 1 \le i \le N-1, \\ u_0 = 2, \qquad u_N = e-1. \end{cases}$$

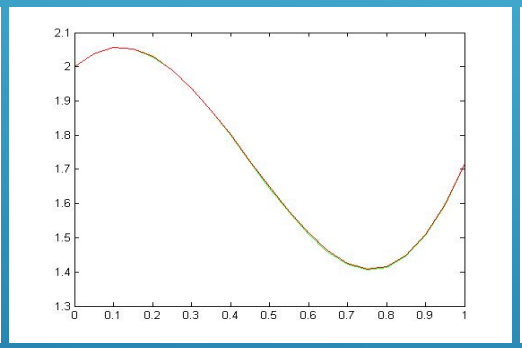
SECTION 3 : APPLICATION

- Two-Point Boundary Value Problem

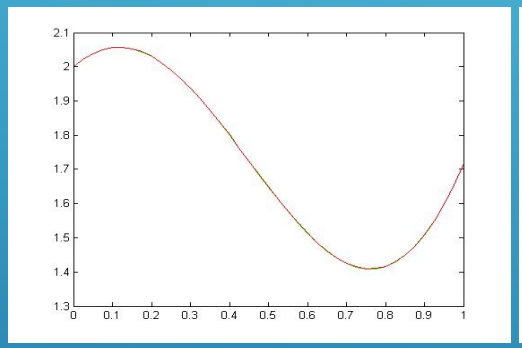
$$AU = F \quad \begin{cases} [2+h(1+x_i)^2]u_{i-1} - [4+2h^2e^{-x_i}]u_i + [2-h(1+x_i)^2]u_{i+1} = 2h^2 f(x_i), \ 1 \le i \le N-1, \\ u_0 = 2, \qquad\qquad u_N = e-1. \end{cases}$$



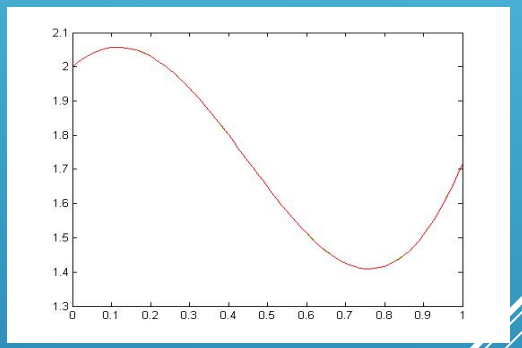
N=10



N=20



N=40



N=80

h	$\|u_h - u\|_c = \max_{1 \le i \le N-1} \|u_i - u(x_i)\|$	$\|u_h - u\|_0 = \sqrt{\sum_{i=1}^{N-1} h(u_i - u(x_i))^2}$
0.1	0.012508648	0.0083
0.05	0.003106943	0.0021
0.025	0.00077705	5.1416e-04
0.0125	1.9420e-04	1.2849e-04